

UNIVERSIDAD ECOTEC

CARRERA DE INGENIERÍA EN SISTEMAS INTELIGENTES

DESARROLLO DE APLICACIONES MÓVILES

Tema: TechStore: consumo de una API REST pública con Providers.

Estudiante: Juan Cristóbal Mera Vásquez

Docente:

MSc. Gregorio Cruz Gonzabay

Quito, 08 de noviembre, 2026

1. Selección y Descripción de la API Pública.

1.1. Justificación de la elección

TechStore nació en la actividad integradora 1 como una aplicación para el registro de componentes de computadora. Para el consumo de API se eligió DummyJSON, ya que ofrece un catálogo de productos tecnológicos acorde con la temática de la aplicación, de modo que la nueva funcionalidad amplía la estructura original en lugar de añadir un módulo ajeno. Además, cumple los requisitos establecidos para la actividad:

- Conexión HTTP: Integración del paquete http (o dio) en Flutter para realizar peticiones asíncronas (GET) a una API pública y gratuita (ej. PokeAPI, CatFact, OpenWeather de acceso libre, u otra API REST sin token de sesión).
- Mapeo de Datos: Conversión correcta de las respuestas en formato JSON a modelos de datos estructurados en Dart (fromJson / toJson).

Elemento	Detalle
API seleccionada	DummyJSON - API REST pública de datos de prueba (https://dummyjson.com)
Endpoint utilizado	https://dummyjson.com/products/category/{categoria}
Categorías consumidas	laptops · tablets · mobile-accessories (tres peticiones en paralelo)
Autenticación	No requiere token, API key ni sesión
Formato de respuesta	JSON (objeto con una lista de productos y datos de paginación)

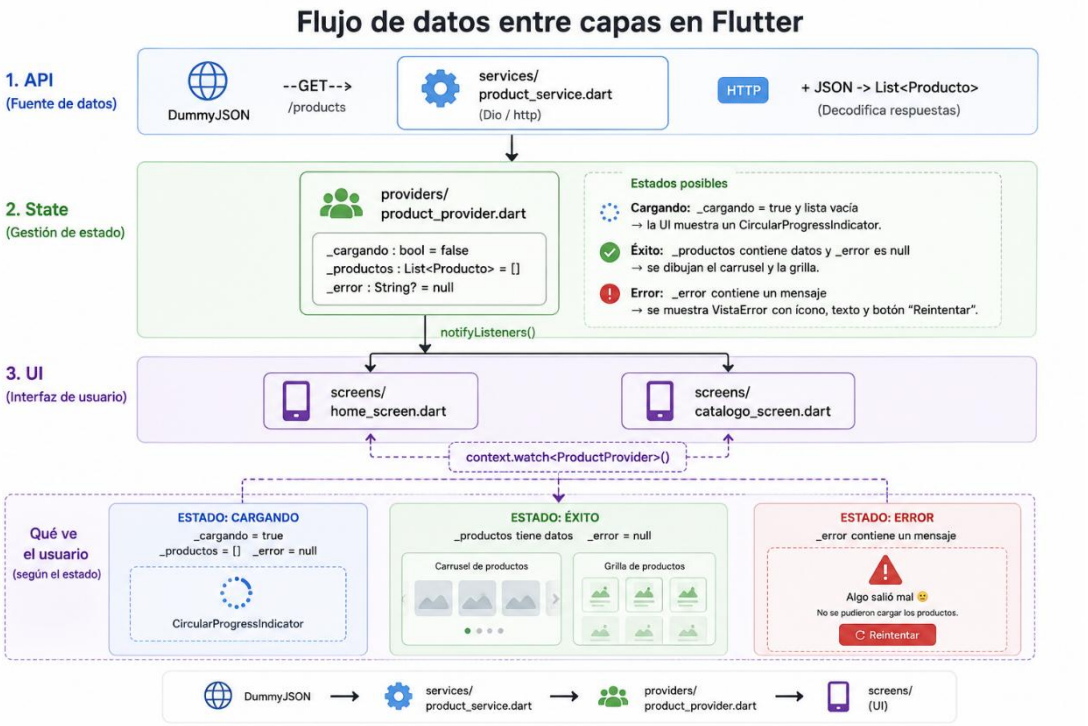
1.2. Estructura del JSON consumido

```
{
  "products": [
    {
      "id":          int,        // → id
      "title":       String,     // → titulo
      "description": String,     // → descripcion
      "category":    String,     // → categoria   ("laptops", "tablets", ...)
      "price":       num,        // → precio      (int o double)
      "rating":      num,        // → rating
      "stock":       int,        // → stock
      "brand":       String?,    // → marca       (puede no venir)
      "thumbnail":   String,     // → imagen      (URL de la miniatura)
      ...                        //    otros campos no utilizados
    }
  ],
  "total": int,
  "skip":  int,
  "limit": int
}
```

2. Arquitectura y Manejo de Estados

La funcionalidad se organizó en 3 capas con responsabilidades independientes:

1. **Capa de servicios (product_service.dart)**: es la única parte de la aplicación que se comunica con la API pública DummyJSON. Realiza las peticiones GET con el paquete http, establece un tiempo máximo de espera, verifica que el servidor responda correctamente (código 200) y convierte el JSON recibido en objetos del modelo Producto mediante fromJson.
2. **Capa de estado (product_provider.dart):** utiliza Provider con ChangeNotifier para administrar los tres estados de la petición: Cargando mientras se espera la respuesta, Éxito cuando los productos llegan correctamente y Error cuando falla la conexión. Cada vez que el estado cambia, el provider llama a notifyListeners() para avisar a la interfaz.
3. **Capa de interfaz (home_screen.dart y catalogo_screen.dart):** las pantallas no importan el paquete http ni conocen la API. Solo escuchan al provider con context.watch y muestran lo que corresponde a cada estado: un indicador de carga, el carrusel y la grilla de productos, o una vista de error con el botón de reintentar.



Capa	Archivo	Responsabilidad
Modelo	models/product_model.dart	Representa un producto; convierte JSON ↔ objeto con fromJson / toJson.
Servicio	services/product_service.dart	Única capa de conexión con la API: GET con timeout de 10 s, validación de statusCode y descarga de las tres categorías en paralelo con Future.wait.
Estado	providers/product_provider.dart	Extiende ChangeNotifier. Guarda productos, carga y mensaje de error; avisa a la UI con notifyListeners().
Interfaz	screens/home_screen.dart, screens/catalogo_screen.dart	Solo dibujan según el estado. Home muestra un carrusel con los 5 mejor valorados; Catálogo, una grilla filtrable.
Widgets	Error_widget.dart, producto_ widget.dart, card_swiper.dart	Componentes reutilizables; VistaError es compartida por ambas pantallas.

3. Extractos de Código Clave

3.1 **Consumo HTTP (product_service.dart)**

```dart
Future<List<Producto>> _obtenerPorCategoria(String categoria) async {
  final url = Uri.parse('$_baseUrl/$categoria');

  // GET con límite de espera: si no responde en 10 s lanza excepción
  final respuesta = await http.get(url).timeout(const Duration(seconds: 10));

  // Cualquier código distinto de 200 se trata como error
  if (respuesta.statusCode ≠ 200) {
    throw Exception('Error HTTP ${respuesta.statusCode} en "$categoria"');
  }

  final Map<String, dynamic> data = jsonDecode(respuesta.body);
  final List<dynamic> lista = data['products'];
  return lista.map((item) ⇒ Producto.fromJson(item)).toList();
}

// Las tres categorías se piden en paralelo y se unen en una sola lista
Future<List<Producto>> obtenerProductos() async {
  final resultados = await Future.wait(
    categorias.map((c) ⇒ _obtenerPorCategoria(c)),
  );
  return resultados.expand((lista) ⇒ lista).toList();
}
```

3.2 Mapeo del modelo JSON (product_model.dart)

```dart
factory Producto.fromJson(Map<String, dynamic> json) => Producto(
      id: json['id'],
      titulo: json['title'],
      descripcion: json['description'],
      categoria: json['category'],
      precio: (json['price'] as num).toDouble(),   // acepta int o double
      rating: (json['rating'] as num).toDouble(),
      marca: json['brand'],                         // nullable
      stock: json['stock'],
      imagen: json['thumbnail'],
    );

Map<String, dynamic> toJson() => {
      'id': id, 'title': titulo, 'description': descripcion,
      'category': categoria, 'price': precio, 'rating': rating,
      'brand': marca, 'stock': stock, 'thumbnail': imagen,
    };
```

3.3 Transición de estados (product_provider.dart)

```dart
Future<void> cargarProductos() async {
  _cargando = true;                // 1) CARGANDO
  _error = null;
  notifyListeners();

  try {
    _productos = await _service.obtenerProductos();   // 2) ÉXITO
  } catch (e) {
    _error = 'No se pudo conectar con el servidor. '
             'Revisa tu conexión a internet.';         // 3) ERROR
    debugPrint('Error al cargar productos: $e');
  } finally {
    _cargando = false;
    notifyListeners();             // la UI se redibuja
  }
}
```

3.4 Manejo de estados en la interfaz (catalogo_screen.dart)

```dart
final provider = context.watch<ProductosProvider>();   // se suscribe al
estado
...
RefreshIndicator(                                       // Pull-to-
Refresh
  onRefresh: provider.cargarProductos,
  child: _contenido(provider),
)

Widget _contenido(ProductosProvider provider) {
  if (provider.cargando && provider.productos.isEmpty) {        //
CARGANDO
    return const Center(child: CircularProgressIndicator());
  }
  if (provider.error != null) {                                 // ERROR
    return ListView(
      physics: const AlwaysScrollableScrollPhysics(),
      children: [
        const SizedBox(height: 80),
        VistaError(mensaje: provider.error!,
                   onReintentar: provider.cargarProductos),
      ],
    );
  }
  final lista = provider.productosFiltrados;                    // ÉXITO
  return GridView.builder(
    physics: const AlwaysScrollableScrollPhysics(),
    itemCount: lista.length,
    gridDelegate: const SliverGridDelegateWithFixedCrossAxisCount(
      crossAxisCount: 2, childAspectRatio: 0.68),
    itemBuilder: (_, index) => ProductoCard(producto: lista[index]),
  );
}
```

4. Evidencias de Pruebas (QA) y Enlace al Repositorio

4.1 Repositorio en GitHub

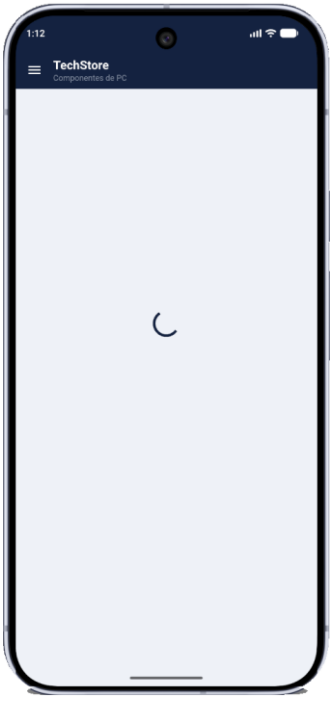
Elemento	Detalle
URL pública	https://github.com/juancmera/flutter_actividad_1
Rama de trabajo	feature/api-integration

4.2 Casos de Pruebas

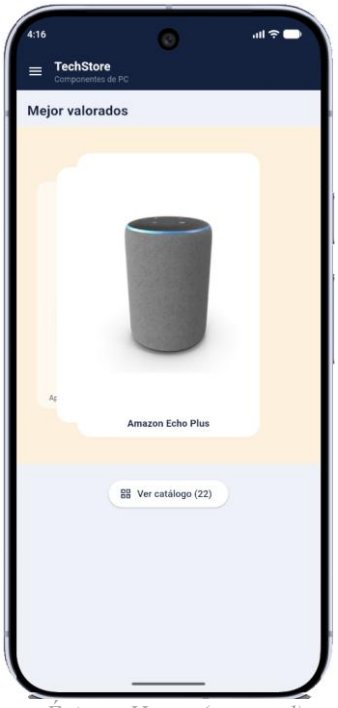
#	Escenario	Procedimiento	Resultado esperado
1	Cargando	Abrir la app (retraso temporal o red limitada en el emulador para captar el estado).	Indicador de progreso circular.

2	Éxito - Home	Abrir la app con conexión.	Carrusel con los productos mejor valorados.
3	Éxito - Catálogo	Ir a Catálogo y usar las categorías.	Tipo filtro por laptops, tablets o accesorios.
4	Error de red	Activar el modo avión y deslizar para actualizar.	Vista de error con mensaje y botón Reintentar.
5	Recuperación	Desactivar el modo avión y pulsar Reintentar.	Los productos vuelven a cargarse.

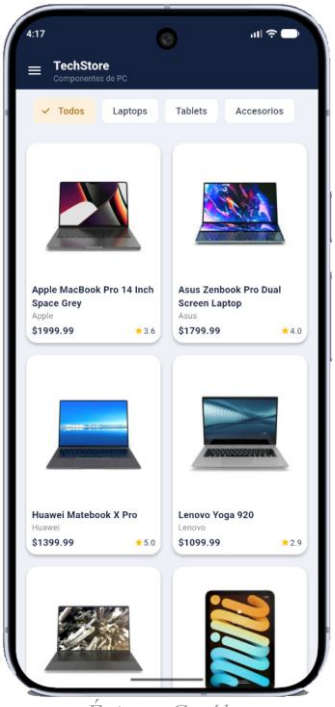
4.3 **Capturas**



Estado de carga



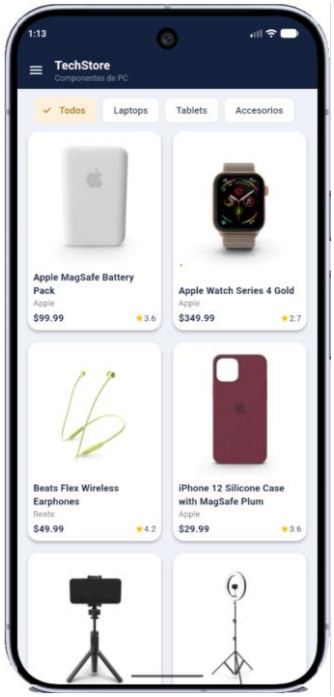
Éxito – Home (carrusel)



Éxito – Catálogo



Error de red (modo avión)



Recuperación tras Reintentar

5. Conclusiones

- Usamos programación asíncrona (async y await), con esto, la app hace la petición y mientras espera la respuesta sigue funcionando, y el usuario visualiza un indicador de carga.
- En un dispositivo móvil, se pierde la señal o se terminan los datos móviles. Por eso la app controla la conexión. Revisa que el servidor responda correctamente (código 200) y fija un tiempo máximo de espera de 10 segundos. Si algo falla, en lugar de cerrarse o quedarse en blanco, muestra el mensaje “No se pudo conectar con el servidor”, y un botón para reintentar la conexión. Así el usuario conoce qué pasó.
- La funcionalidad se organizó en 3 capas con responsabilidades independientes. El servicio se conecta a la API, el Provider guarda los datos y el estado, y las pantallas solo muestran la información. Gracias a esto, las pantallas de Inicio y Catálogo usan los mismos datos sin pedirlos dos veces a la API, y si más adelante se cambia, solo hay que modificar el servicio, sin tocar las pantallas.

Una app conectada a internet no solo debe funcionar cuando todo sale bien. También debe saber qué mostrar mientras carga y qué hacer cuando falla la conexión. Por eso se manejaron los tres estados, Cargando, Éxito y Error, que hacen que la app sea confiable en situaciones reales.

6. Declaración del Uso de Inteligencia Artificial (IA)

En el desarrollo de la Actividad Integradora 3, utilicé el asistente Claude (Anthropic) como prueba del código base de provider y pantallas, que luego integré, adapté y probé en mi proyecto, además de elaborar la infografía del flujo de datos.